

Number Theory

CS1010 Discrete Mathematics for Computer Science

Shibam Ghosh

August 27, 2026

Department of CSE, IIT Hyderabad



Integer Representations

Why Care About Integer Representations?

The Human Way: Base 10

- ▶ We favor **decimal** (base 10) notation (likely because we have 10 fingers).
 - ▶ Every digit represents a power of 10: $125 = 1 \cdot 10^2 + 2 \cdot 10^1 + 5 \cdot 10^0$
-

The Computer Way: Alternative Bases

- ▶ Mathematically, we can use **any base** $b > 1$.
- ▶ For computers, 10 fingers don't matter; electrical switches (on/off) do.
- ▶ This makes three specific bases critical for computer science:
 - ▷ **Binary** ($b = 2$): The literal language of hardware.
 - ▷ **Octal** ($b = 8$) & **Hexadecimal** ($b = 16$): (Human-readable shorthand)



Why Care About Integer Representations?

The Human Way: Base 10

- ▶ We favor **decimal** (base 10) notation (likely because we have 10 fingers).
 - ▶ Every digit represents a power of 10: $125 = 1 \cdot 10^2 + 2 \cdot 10^1 + 5 \cdot 10^0$
-

The Computer Way: Alternative Bases

- ▶ Mathematically, we can use **any base** $b > 1$.
- ▶ For computers, 10 fingers don't matter; electrical switches (on/off) do.
- ▶ This makes three specific bases critical for computer science:
 - ▷ **Binary** ($b = 2$): The literal language of hardware.
 - ▷ **Octal** ($b = 8$) & **Hexadecimal** ($b = 16$): (Human-readable shorthand)



Base b Representations

Theorem

Let b be a positive integer greater than 1. If n is a positive integer, it can be expressed uniquely in the form:

$$n = a_k b^k + a_{k-1} b^{k-1} + \dots + a_1 b + a_0$$

where k is a nonnegative integer, $a_0, a_1, \dots, a_k$ are nonnegative integers less than b , and $a_k \neq 0$.

- ▶ The a_j values are called the **base-b digits**.
- ▶ The string representation is denoted by $(a_k a_{k-1} \dots a_1 a_0)_b$.
- ▶ Subscript 10 is usually omitted for standard decimal integers.



Comparing Hexadecimal, Octal, and Binary

Dec	0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15
Hex	0	1	2	3	4	5	6	7	8	9	A	B	C	D	E	F
Oct	0	1	2	3	4	5	6	7	10	11	12	13	14	15	16	17
Bin	0	1	10	11	100	101	110	111	1000	1001	1010	1011	1100	1101	1110	1111

- ▶ Each octal digit corresponds to a block of 3 binary digits.
- ▶ Each hexadecimal digit corresponds to a block of 4 binary digits.

Examples

- ▶ What is the decimal expansion of $(1\ 0110\ 1101)_2$?
- ▶ $1 \cdot 2^8 + 0 \cdot 2^7 + 1 \cdot 2^6 + 1 \cdot 2^5 + 0 \cdot 2^4 + 1 \cdot 2^3 + 1 \cdot 2^2 + 0 \cdot 2^1 + 1 \cdot 2^0$
 $= 256 + 0 + 64 + 32 + 0 + 8 + 4 + 0 + 1 = 365$

Examples

- ▶ What is the decimal expansion of $(1\ 0110\ 1101)_2$?
- ▶ $1 \cdot 2^8 + 0 \cdot 2^7 + 1 \cdot 2^6 + 1 \cdot 2^5 + 0 \cdot 2^4 + 1 \cdot 2^3 + 1 \cdot 2^2 + 0 \cdot 2^1 + 1 \cdot 2^0$
 $= 256 + 0 + 64 + 32 + 0 + 8 + 4 + 0 + 1 = 365$
- ▶ What is the decimal expansion of $(10110)_2$?

Examples

- ▶ What is the decimal expansion of $(1\ 0110\ 1101)_2$?
- ▶ $1 \cdot 2^8 + 0 \cdot 2^7 + 1 \cdot 2^6 + 1 \cdot 2^5 + 0 \cdot 2^4 + 1 \cdot 2^3 + 1 \cdot 2^2 + 0 \cdot 2^1 + 1 \cdot 2^0$
 $= 256 + 0 + 64 + 32 + 0 + 8 + 4 + 0 + 1 = 365$
- ▶ What is the decimal expansion of $(10110)_2$?
- ▶ $1 \cdot 2^4 + 0 \cdot 2^3 + 1 \cdot 2^2 + 1 \cdot 2^1 + 0 \cdot 2^0 = 16 + 0 + 4 + 2 + 0 = 22$

Examples

- ▶ What is the decimal expansion of $(1\ 0110\ 1101)_2$?
- ▶ $1 \cdot 2^8 + 0 \cdot 2^7 + 1 \cdot 2^6 + 1 \cdot 2^5 + 0 \cdot 2^4 + 1 \cdot 2^3 + 1 \cdot 2^2 + 0 \cdot 2^1 + 1 \cdot 2^0$
 $= 256 + 0 + 64 + 32 + 0 + 8 + 4 + 0 + 1 = 365$
- ▶ What is the decimal expansion of $(10110)_2$?
- ▶ $1 \cdot 2^4 + 0 \cdot 2^3 + 1 \cdot 2^2 + 1 \cdot 2^1 + 0 \cdot 2^0 = 16 + 0 + 4 + 2 + 0 = 22$
- ▶ What is the decimal expansion of $(5037)_8$?
- ▶ $5 \cdot 8^3 + 0 \cdot 8^2 + 3 \cdot 8^1 + 7 \cdot 8^0 = 5(512) + 0 + 3(8) + 7(1) = 2591$

Examples

- ▶ What is the decimal expansion of $(1\ 0110\ 1101)_2$?
- ▶ $1 \cdot 2^8 + 0 \cdot 2^7 + 1 \cdot 2^6 + 1 \cdot 2^5 + 0 \cdot 2^4 + 1 \cdot 2^3 + 1 \cdot 2^2 + 0 \cdot 2^1 + 1 \cdot 2^0$
 $= 256 + 0 + 64 + 32 + 0 + 8 + 4 + 0 + 1 = 365$
- ▶ What is the decimal expansion of $(10110)_2$?
- ▶ $1 \cdot 2^4 + 0 \cdot 2^3 + 1 \cdot 2^2 + 1 \cdot 2^1 + 0 \cdot 2^0 = 16 + 0 + 4 + 2 + 0 = 22$
- ▶ What is the decimal expansion of $(5037)_8$?
- ▶ $5 \cdot 8^3 + 0 \cdot 8^2 + 3 \cdot 8^1 + 7 \cdot 8^0 = 5(512) + 0 + 3(8) + 7(1) = 2591$
- ▶ What is the decimal expansion of $(1C0FA)_{16}$?
- ▶ $1 \cdot 16^4 + 12 \cdot 16^3 + 0 \cdot 16^2 + 15 \cdot 16^1 + 10 \cdot 16^0$
 $= 65536 + 12(4096) + 0 + 15(16) + 10(1) = 114938$

Constructing Base b Expansions

1. Let $q := n$
2. Let $k := 0$
3. While $q \neq 0$ do:
4. $a_k := q \bmod b$
5. $q := q/b$
6. $k := k + 1$
7. $(a_{k-1}, \dots, a_1, a_0)$ is the
Base b expansion of n

Find the octal expansion of $(9876)_{10}$.

$$9876 = 8 \cdot 1234 + 4 \quad (\text{Remainder: } 4) \uparrow$$

$$1234 = 8 \cdot 154 + 2 \quad (\text{Remainder: } 2) |$$

$$154 = 8 \cdot 19 + 2 \quad (\text{Remainder: } 2) |$$

$$19 = 8 \cdot 2 + 3 \quad (\text{Remainder: } 3) |$$

$$2 = 8 \cdot 0 + 2 \quad (\text{Remainder: } 2) |$$

Reading the remainders from bottom to top:

$$(23224)_8.$$

Integer Representations

Side note on Algorithm

What is an Algorithm?

Definition

An **algorithm** is a finite set of precise instructions for performing a computation or for solving a problem.

Why review this now?

- ▶ While you will dive deeply into implementation details in your *Introduction to Programming* course, we need a baseline understanding here.
- ▶ In discrete mathematics, we treat algorithms as mathematical objects to analyze their logic, correctness, and efficiency.



Describing an Algorithm: Finding the Maximum

example

Describe a step-by-step procedure for finding the maximum value in a finite sequence of integers.

Solution (Steps in Plain Human Language):

1. Set the *temporary maximum* equal to the first integer in the sequence.
2. Compare the next integer in the sequence to the temporary maximum.
3. **If** it is larger, update the temporary maximum to be this new integer.
4. **Repeat** the previous steps if there are more integers left. If not, stop.

When the algorithm terminates, the temporary maximum is the largest integer in the sequence.



Specifying Algorithms: Pseudocode

What is Pseudocode?

- ▶ It is an intermediate step between an English language description and actual programming code (like Python or C++).
- ▶ Programmers use pseudocode as a blueprint to confidently construct programs in any language.

Pseudocode for Finding the Maximum

find_max([$a_0, a_1, a_2, \dots, a_n$])

1. $max := a_0$
2. **for** $i := 1$ **to** n
3. **if** $a_i > max$ **then**
4. $max := a_i$
5. **return** max
6. **end** *find_max*

Core Properties of All Algorithms

To be mathematically sound, any algorithm must satisfy these key properties:

Input & Output: Accepts input values from a specified set and produces a solution.

► Think of an algorithm as a function

Correctness: It must produce the correct output values for *each* valid set of input values.

Finiteness: It must terminate after a finite number of steps for any input.

Effectiveness: Each step must be clear and performable in a finite amount of time.



Binary Addition

Adding Binary Numbers

Inputs:

$$a = (a_{n-1} \dots a_1 a_0)_2$$

$$b = (b_{n-1} \dots b_1 b_0)_2.$$

-
- ▶ Set $c \leftarrow 0$.
 - ▶ For $j = 0, \dots, n-1$:
 - ▷ Compute $t = a_j + b_j + c$.
 - ▷ Set $s_j = t \bmod 2$.
 - ▷ Set $c = \lfloor t/2 \rfloor$.
 - ▶ Finally, set $s_n \leftarrow c$.

Return: $(s_n s_{n-1} \dots s_1 s_0)_2$.

Example

Add $(1011)_2 + (1101)_2$.

$$\begin{array}{r} 1011 \\ +1101 \\ \hline 11000 \end{array}$$

- ▶ $1 + 1 = 10_2$: write 0, carry 1.
- ▶ $1 + 0 + 1 = 10_2$: write 0, carry 1.
- ▶ $0 + 1 + 1 = 10_2$: write 0, carry 1.
- ▶ $1 + 1 + 1 = 11_2$: write 1, carry 1.

Binary Multiplication

Algorithm

Inputs:

$$a = (a_{n-1} \dots a_1 a_0)_2$$

$$b = (b_{n-1} \dots b_1 b_0)_2.$$

-
- ▶ For $j = 0, \dots, n - 1$:
 - ▷ If $b_j = 1$, set $c_j = a \ll j$.
 - ▷ If $b_j = 0$, set $c_j = 0$.
 - ▶ Initialize $p \leftarrow 0$.
 - ▶ For $j = 0, \dots, n - 1$: $p \leftarrow \text{add}(p, c_j)$.
 - ▶ Return p .

Example

Multiply $(110)_2(101)_2$.

$$\begin{array}{r} 110 \\ \times 101 \\ \hline 110 \\ 000 \\ 110 \\ \hline 11110 \end{array}$$

Hence,

$1011_2 \cdot 1101_2 = 10001111_2.$

Binary Exponentiation

Square-and-Multiply

Inputs: a and $e = (e_{n-1} \dots e_1 e_0)_2$

- ▶ Set $y \leftarrow 1$.
- ▶ For $i = n - 1, \dots, 0$:
 - ▷ Square: $y \leftarrow y^2$.
 - ▷ If $e_i = 1$, multiply: $y \leftarrow y \cdot a$.
- ▶ Return y .

Thus, each bit requires one **squaring**, and a 1-bit requires one additional **multiplication**.

Example

Compute a^{13} .

Since $13 = (1101)_2$, process the bits from left to right:

e_i	Operation	y
1	$1^2 \cdot a$	a
1	$a^2 \cdot a$	a^3
0	$(a^3)^2$	a^6
1	$(a^6)^2 \cdot a$	a^{13}

Hence,

$$a^{13} = (((a^2)a)^2)^2 a.$$



Prime Numbers

Prime Numbers

Definition

A positive integer $p > 1$ is called **prime** if the only positive factors of p are **1** and **p** . A positive integer greater than 1 that is not prime is called **composite**.

The Fundamental Theorem of Arithmetic

Every positive integer greater than 1 can be written uniquely as a prime or as the product of two or more primes where the prime factors are written in order of nondecreasing size.

- ▶ $100 = 2 \cdot 2 \cdot 5 \cdot 5 = 2^2 \cdot 5^2$
- ▶ $32 = 2 \cdot 2 \cdot 2 \cdot 2 \cdot 2 = 2^5$
- ▶ $37 = 37$ (Prime)



Prime Numbers

Definition

A positive integer $p > 1$ is called **prime** if the only positive factors of p are **1** and **p** . A positive integer greater than 1 that is not prime is called **composite**.

The Fundamental Theorem of Arithmetic

Every positive integer greater than 1 can be written uniquely as a prime or as the product of two or more primes where the prime factors are written in order of nondecreasing size.

- ▶ $100 = 2 \cdot 2 \cdot 5 \cdot 5 = 2^2 \cdot 5^2$
- ▶ $32 = 2 \cdot 2 \cdot 2 \cdot 2 \cdot 2 \cdot 2 = 2^5$
- ▶ $37 = 37$ (Prime)



Mersenne Primes

Definition

Prime numbers of the form $2^p - 1$, where p is prime, are called **Mersenne primes**.

- ▶ $2^2 - 1 = 3$, $2^3 - 1 = 7$, $2^5 - 1 = 31$, $2^7 - 1 = 127$ are Mersenne primes.
- ▶ $2^{11} - 1 = 2047$ is **not** prime ($2047 = 23 \cdot 89$).
- ▶ The largest known primes are Mersenne primes tested via the **GIMPS** distributed computing project (<http://www.mersenne.org/>).



Important Results on Prime Numbers

Euclid's Theorem on Primes

There are infinitely many primes.

Prime Number Theorem

The ratio of the number of primes not exceeding x and $x / \ln x$ approaches 1 as x grows without bound.

- ▶ The number of primes $\leq x$ is approximated well by $x / \ln x$.
- ▶ The odds that a randomly selected positive integer less than n is prime are approximately $1 / \ln n$.

Trial Division

Algorithm

Input: An integer $n > 1$.

- ▶ For $d = 2, 3, \dots, \lfloor \sqrt{n} \rfloor$:
 - ▷ If $d \mid n$, return **composite**.
- ▶ Return **prime**.

Example

Is 97 prime?

- ▶ $\sqrt{97} < 10$.
- ▶ Test $d = 2, 3, 4, \dots, 9$.
- ▶ None divides 97.

Hence, 97 is prime.

Why stop at $\sqrt{n}$?

If $n = ab$ is composite, then at least one of a, b is at most $\sqrt{n}$.



Generating Primes

- ▶ In cryptography, finding massive primes with hundreds of digits is essential.
- ▶ No simple closed formula exists that always produces primes.
- ▶ $f(n) = n^2 - n + 41$ produces primes for $n = 1, 2, \dots, 40$, but fails at $f(41) = 41^2$.
- ▶ **Theorem:** No integer-coefficient polynomial exists that outputs only primes for all positive integers.
- ▶ Fortunately, we can generate massive integers that are *probabilistically* almost certain to be prime.

Unresolved Conjectures about Primes

- ▶ **Goldbach's Conjecture:** Every even integer $n > 2$ is the sum of two primes.
Verified up to $1.6 \cdot 10^{18}$.
- ▶ There exist infinitely many primes of the form $n^2 + 1$.
- ▶ **Twin Prime Conjecture:** There are infinitely many pairs of twin primes (primes that differ by 2, like 3 and 5, 5 and 7, 11 and 13).



Thank You for your
attention!

Any questions?